

# Rapport technique — Système RAG de recommandation de films

## Auteur

Rodrigue

---

## 1. Présentation du projet

Ce projet implémente un système de recommandation de films basé sur une approche **RAG (Retrieval-Augmented Generation)**.

Le système permet :

- de transformer les films en embeddings avec SentenceTransformer
- d'effectuer une recherche vectorielle
- de récupérer les films les plus pertinents
- de générer une réponse avec un LLM via Groq

Le projet a été développé en Python.

---

## 2. Architecture du pipeline RAG

Le pipeline se déroule en plusieurs étapes :

### Étape 1 — Préparation des données

Les données du dataset sont transformées en documents structurés contenant :

- le titre
- les genres
- la date
- la note
- le résumé
- la langue

Exemple de structure :

```
{  
  "contenu": "Titre: Inception ...",  
  "metadata": {  
    "titre": "Inception",  
    "note": 8.2,  
  },  
}
```

```
}  
    "langue": "en"  
}
```

---

## Étape 2 — Génération des embeddings

Les embeddings sont générés avec le modèle :

```
SentenceTransformer("distiluse-base-multilingual-cased-v2")
```

Ce modèle a été choisi car il fonctionne correctement avec des contenus multilingues.

---

## Étape 3 — Recherche vectorielle

Le projet utilise FAISS pour la création de la base vectorielle.

Les embeddings sont sauvegardés afin d'éviter de recalculer l'ensemble des vecteurs à chaque exécution.

---

## Étape 4 — Génération de réponse

Les chunks les plus pertinents sont envoyés au modèle Groq afin de générer une réponse contextualisée.

Le modèle utilisé est :

```
llama-3.3-70b-versatile
```

---

# 3. Difficultés rencontrées

## Problème avec FAISS (segmentation fault)

Lors de l'utilisation de FAISS pour la recherche vectorielle ( `index.search` ), une erreur de type **segmentation fault** s'est produite sur macOS (Apple Silicon).

### Causes possibles

- incompatibilité entre FAISS et certaines architectures ARM
- incompatibilité avec NumPy 2.x

## Problème de compatibilité NumPy

Une erreur indiquait que FAISS n'était pas compatible avec NumPy 2.x.

### Solution

Utilisation d'une version inférieure de NumPy :

```
pip install "numpy<2"
```

---

## Contournement du bug FAISS

FAISS a été conservé pour la phase d'indexation.

Cependant, la fonction `index.search()` provoquant des crashes, une solution alternative a été mise en place.

### Solution utilisée

- récupération des embeddings
- calcul manuel de similarité avec NumPy
- produit scalaire entre les vecteurs
- tri des scores pour obtenir les top-k résultats

### Avantages

- respect des consignes du TP
- stabilité sur macOS
- résultats exploitables malgré les limitations de FAISS

---

## Problème de taille du dataset

Au début du projet, le dataset était limité à 500 films afin d'accélérer les tests.

```
documents = documents[:500]
```

Cependant, cela a entraîné plusieurs problèmes :

- faible diversité des résultats
- très peu de films français disponibles
- recommandations peu pertinentes
- impression que le filtre de langue ne fonctionnait pas

Après analyse, il s'est avéré que le problème ne venait pas du code mais du manque de données.

## Solution

- suppression de la limitation
- utilisation de l'ensemble du dataset (~4800 films)

## Résultats obtenus

- augmentation de la diversité des recommandations
  - amélioration de la pertinence des résultats
  - validation correcte du filtre de langue
- 

## Problème de modèle Groq

Le modèle initial utilisé :

```
llama3-8b-8192
```

n'était plus disponible.

## Solution

Utilisation du modèle :

```
llama-3.3-70b-versatile
```

---

## Évolution du prompt

Au début du projet, un fichier `context.txt` était utilisé pour construire le prompt.

Cette approche a ensuite été remplacée par un prompt directement intégré dans le code.

## Raisons du changement

- meilleur contrôle du comportement du modèle
- simplification du projet
- meilleure adaptation au système de recommandation

Le fichier `context.txt` a donc été supprimé dans la version finale.

---

# 4. Optimisation du chargement des ressources

Une optimisation a été ajoutée dans la version finale afin d'éviter le rechargement des ressources à chaque question utilisateur.

## Avant optimisation

Les éléments suivants étaient rechargés à chaque requête :

- client Groq
- modèle SentenceTransformer
- embeddings
- index vectoriel

Cela entraînait :

- des temps de réponse plus élevés
  - des chargements inutiles
  - une consommation mémoire plus importante
- 

## Solution mise en place

Les ressources sont désormais chargées une seule fois au démarrage du programme.

Les objets sont ensuite réutilisés pendant toute l'exécution du système.

### Avantages

- amélioration des performances
  - comportement plus stable
  - architecture plus propre
  - fonctionnement plus proche d'un système RAG réel
- 

## 5. Résultats obtenus

Le système permet :

- de recommander des films à partir d'une requête utilisateur
  - de générer des réponses basées sur le contexte récupéré
  - d'éviter les hallucinations grâce à des contraintes dans le prompt
  - de filtrer certains résultats selon la langue
- 

## Questions du TP

### Q1

Structuration des données avec :

- titre
- genres

- résumé
  - note
  - langue
- 

## Q2

Utilisation de `json.loads()` pour parser les genres.

---

## Q3

Sauvegarde des embeddings et de la base vectorielle afin d'éviter les recalculs.

---

## Q4

Création d'un prompt système contrôlé afin de limiter les hallucinations.

---

## Q5

Le système répond :

Je ne sais pas

lorsqu'aucune information pertinente n'est trouvée.

---

# 6. Exemple d'utilisation

## Requête utilisateur

films similaires à Inception

---

## Réponse générée

Le système retourne :

- des recommandations de films
  - les genres
  - les notes
  - une description courte
-

## 7. Dataset

Le dataset n'est pas inclus dans le repository GitHub.

Pour exécuter le projet, le fichier CSV doit être placé dans :

```
Data/movies.csv
```

---

## 8. Conclusion

Ce projet a permis de mettre en pratique les différentes étapes nécessaires à la création d'un système RAG :

- préparation des données
- embeddings
- recherche vectorielle
- génération de réponses avec un LLM
- optimisation des performances

Malgré plusieurs difficultés techniques liées à FAISS et à macOS Apple Silicon, une solution fonctionnelle et stable a pu être mise en place.

Le système final est capable de fournir des recommandations contextualisées de manière interactive via le terminal.